

Raport - zadanie 1

Problem komiwojażera (TSP) - RAND, NN, RNN oraz brute-force

Imię i nazwisko: Robert Tworek

Numer indeksu: 276272

Przedmiot: Projektowanie Efektywnych Algorytmów

1. Treść zadania, cel i hipotezy

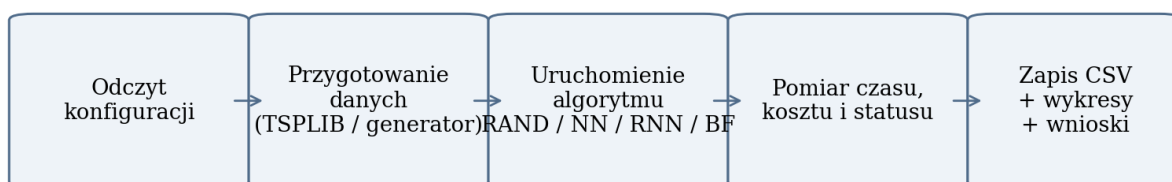
W zadaniu 1 miałem zaimplementować i zbadać cztery metody rozwiązywania problemu komiwojażera: algorytm losowy RAND, algorytm zachłanny Nearest Neighbour (NN), algorytm Repetitive Nearest Neighbour (RNN) oraz metodę przeglądu zupełnego brute-force. Dla każdej metody miałem wyznaczyć zależność czasu uzyskania rozwiązania od rozmiaru instancji, ocenić jakość wyniku oraz oszacować złożoność obliczeniową.

Celem pracy było porównanie dokładności i czasu działania badanych algorytmów oraz sprawdzenie, kiedy rozwiązania przybliżone są praktycznie korzystniejsze od metody dokładnej. Przyjąłem hipotezy, że brute-force zagwarantuje optimum, ale szybko stanie się niepraktyczny; RAND będzie szybki, lecz mało dokładny; NN będzie dobrym kompromisem między czasem i jakością; a RNN poprawi wyniki NN kosztem wyraźnie dłuższego czasu działania. Klasyczną definicję TSP i rolę benchmarków TSPLIB oparłem na literaturze [1]-[3].

2. Metoda, implementacja, dane i konfiguracja

Badałem klasyczny TSP, czyli wyznaczanie cyklu Hamiltona o minimalnym koszcie [2][3]. Do badań heurystycznych wykorzystałem instancje TSPLIB [1]: eil51, berlin52, st70, eil76, kroA100, rd100, bier127 oraz ch130. Do badań brute-force wygenerowałem pełne grafy ważone w dwóch wariantach: symetrycznym i asymetrycznym, dla $n = 6..15$, po 10 instancji każdego typu i rozmiaru.

Program przygotowałem jako jedną samodzielną aplikację sterowaną plikiem konfiguracyjnym. W RAND generowałem wiele losowych permutacji i wybierałem najlepszą; w NN w każdym kroku przechodziłem do najbliższego nieodwiedzonego wierzchołka; w RNN uruchamiałem NN dla różnych wierzchołków startowych, a przy remisach rozważałem wszystkie równorzędne wybory, zgodnie z treścią zadania; w brute-force przeglądałem wszystkie dopuszczalne permutacje przy ustalonym wierzchołku startowym.



Rysunek 1. Schemat działania programu wykorzystanego przeze mnie w zadaniu 1.

Tabela 1. Najważniejsze pola pliku konfiguracyjnego wraz z przykładowymi wartościami.

Parametr	Znaczenie	Przykład BF	Przykład heurystyk
mode	wybór trybu pracy	generated	tsplib
algorithms	lista algorytmów	bruteforce	rand, nn, rnn
timeout_seconds	limit czasu [s]	5	10
output	plik CSV z wynikami	bf_generated.csv	heuristics_tsplib_fixed.csv
dane wejściowe	zakres / ścieżka	sizes=6..15	data/tsplib/*.tsp

Badania przeprowadziłem w dwóch etapach. Najpierw uruchomiłem brute-force na danych wygenerowanych i dla każdej instancji mierzyłem czas oraz status zakończenia przed limitem 5 s. Następnie przebywałem RAND, NN i RNN na instancjach TSPLIB, zapisując koszt trasy, czas działania, błąd względny, informacje o ukończeniu oraz timeoutach.

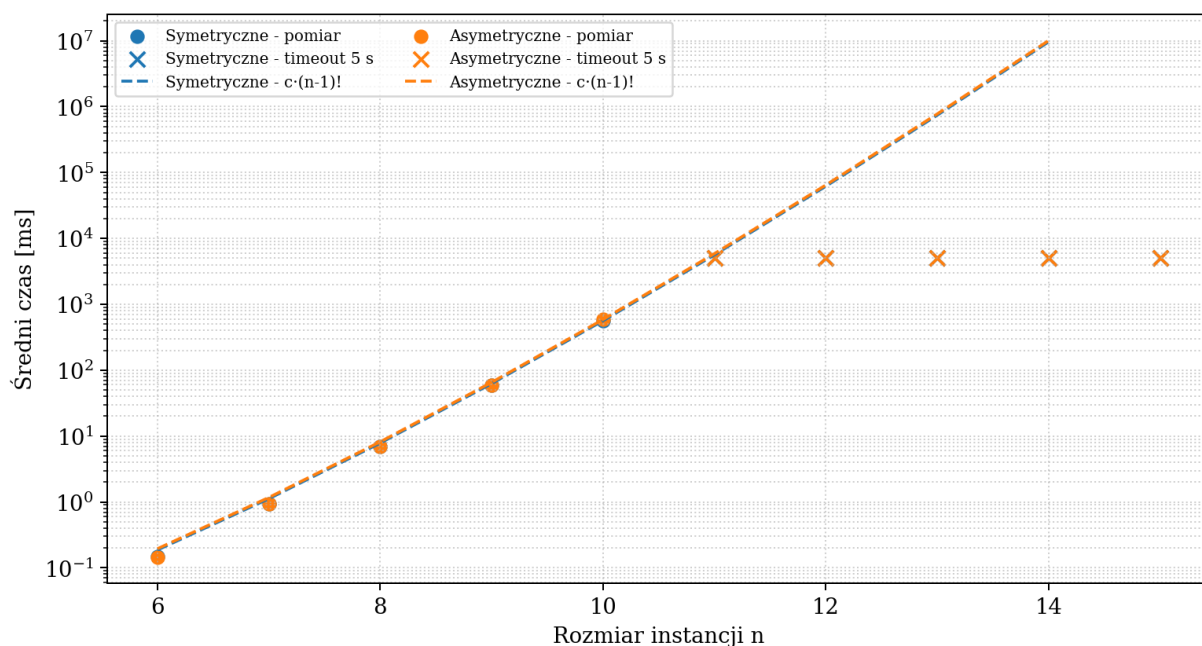
3. Wyniki badań dla brute-force

Wyniki brute-force potwierdziły bardzo gwałtowny wzrost czasu działania wraz ze wzrostem liczby miast. Dla $n = 6..10$ algorytm kończył pracę dla wszystkich instancji, natomiast od $n = 11$ wzwyż każda instancja kończyła się przekroczeniem limitu 5 s. Oznacza to, że już w tym zakresie widoczny jest faktorialny charakter złożoności.

Tabela 2. Średni czas działania brute-force oraz liczba ukończonych instancji.

Typ grafu	n	Średni czas [ms]	Ukończone	Timeouty
Symetryczny	6	0.1	10	0
Symetryczny	8	7.0	10	0
Symetryczny	10	564.3	10	0
Symetryczny	11	5000.0	0	10
Asymetryczny	6	0.1	10	0
Asymetryczny	8	7.0	10	0
Asymetryczny	10	589.1	10	0
Asymetryczny	11	5000.0	0	10

Dla $n = 11$ wartość 5000.0 ms odpowiada osiągnięciu limitu czasu, a nie czasowi pełnego rozwiązania. Na podstawie dopasowania funkcji $c \cdot (n-1)!$ do danych dla $n = 6..10$ oszacowałem, że dla $n = 13$ średni czas wynosi ok. 12.7 min, natomiast dla $n = 14$ przekracza ok. 2.7 h.



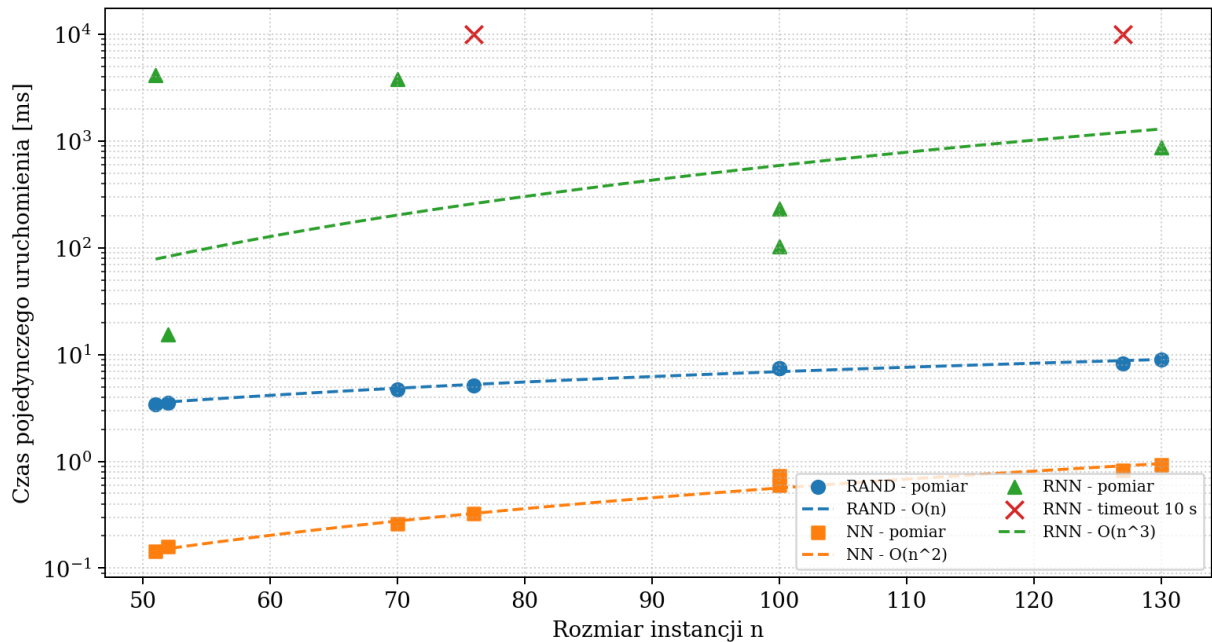
Rysunek 2. Zależność czasu działania brute-force od rozmiaru instancji; punkty przedstawiają pomiary, a linie przerywane dopasowanie $c \cdot (n-1)!$.

4. Wyniki badań dla RAND, NN i RNN

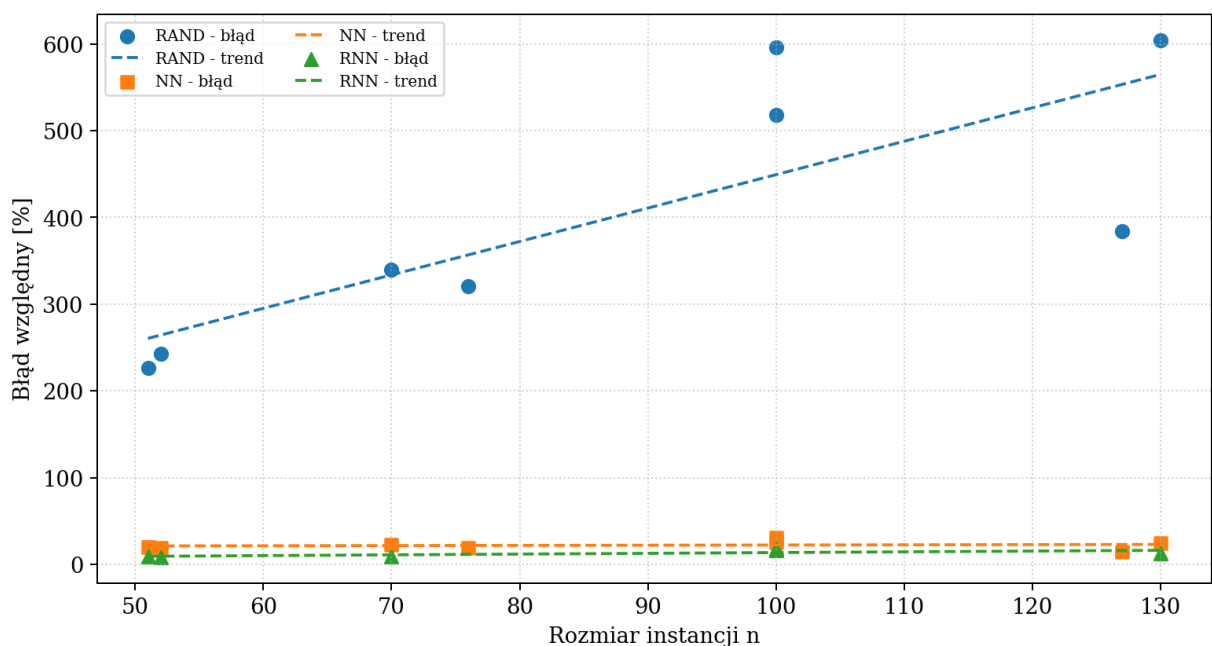
W części heurystycznej porównałem trzy metody: RAND, NN i RNN. RAND był szybki, ale generował słabe jakościowo trasy. NN działał najszybciej i dawał wyraźnie lepsze wyniki. RNN uzyskiwał najlepszą jakość spośród heurystyk, ale kosztem znacznie większego czasu działania; dla dwóch instancji przekroczył limit 10 s.

Tabela 3. Zestawienie średnich wyników dla heurystyk na instancjach TSPLIB.

Algorytm	Średni błąd względny [%]	Średni czas [ms]	Ukończone	Timeouty
RAND	404.0	6.1	8	0
NN	22.1	0.5	8	0
RNN	12.2	3653.0	6	2



Rysunek 3. Czas pojedynczego uruchomienia RAND, NN i RNN w funkcji rozmiaru instancji; punkty oznaczają pomiary, a linie przerywane trendy zgodne z oszacowaną złożonością.



Rysunek 4. Błąd względny RAND, NN i RNN dla instancji TSPLIB; punkty oznaczają pomiary, a linie przerywane trendy zmian.

5. Analiza wyników, odpowiedzi na pytania i wnioski

Uzyskane wyniki potwierdziły wszystkie postawione hipotezy. Brute-force jest metodą referencyjną, ale praktycznie użyteczną tylko dla bardzo małych instancji. Przejście z $n = 10$ do $n = 11$ powoduje wejście w obszar timeoutów, a estymacja z rysunku 2 wskazuje, że granica praktyczna przy limicie 30 minut wypada około $n = 13$. Dla $n = 14$ czas jest już wyraźnie większy od rozsądnego limitu.

W przypadku heurystyk widoczny jest kompromis między czasem i jakością. RAND ma bardzo niski koszt obliczeń, ale średni błąd względny wynosi aż 404.0%. NN działa praktycznie natychmiast, a jego błąd średni spada do 22.1%. RNN daje najlepszą jakość spośród badanych heurystyk (12.2%), ale cena za to jest wysoka: średni czas 3653.0 ms oraz dwa timeouty. Zgodnie z wykresem z rysunku 3 ograniczenie 15 minut dla heurystyk jest uzasadnione głównie dla RNN, ponieważ RAND i NN rosną odpowiednio jak $O(k \cdot n)$ i $O(n^2)$, natomiast RNN - co najmniej jak $O(n^3)$, a przy remisach szybciej.

Na pytania interpretacyjne odpowiadam następująco. RNN jest korzystniejszy od NN wtedy, gdy ważniejsza jest jakość rozwiązania niż czas, ponieważ wykorzystanie wielu startów zmniejsza ryzyko złego wyboru lokalnego. W algorytmie losowym można ograniczać liczbę powtórzeń permutacji przez zapamiętywanie wcześniej wygenerowanych tras; rozwiązanie zachowuje losowy charakter, ale zwiększa koszt pamięci. W moich badaniach wykorzystywałem grafy pełne, więc gęstość grafu była stała; przy wielu ścieżkach optymalnych heurystyka ma większą szansę trafić wynik bliski optimum; szeroki zakres wag zmniejsza liczbę remisów, a wąski może je zwiększać, co najmocniej obciąża RNN. Górne ograniczenie (UB) uzyskiwałem bezpośrednio z kosztu dowolnej poprawnej trasy, natomiast dolne ograniczenie (LB) ma największe znaczenie w metodach dokładnych, np. branch-and-bound.

Ostatecznie stwierdzam, że nie istnieje jedna metoda najlepsza pod każdym względem. Brute-force zapewnia optimum, ale szybko staje się niepraktyczny. RAND jest prosty i szybki, lecz bardzo niedokładny. NN okazał się najszybszym i najbardziej praktycznym algorytmem bazowym. RNN uzyskał najlepszą jakość spośród badanych heurystyk, ale wymagał wyraźnie większego czasu. Najważniejszym wnioskiem z pracy jest to, że w TSP poprawa jakości rozwiązania zwykle odbywa się kosztem czasu działania.

Bibliografia

- [1] G. Reinelt, "TSPLIB - A Traveling Salesman Problem Library", ORSA Journal on Computing, 3(4), 1991.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, Wprowadzenie do algorytmów, WNT, Warszawa.
- [3] D. L. Applegate, R. E. Bixby, V. Chvátal, W. J. Cook, The Traveling Salesman Problem: A Computational Study, Princeton University Press, 2006.